CS24301 — THEORY (3.0 CR) | MODULE I NOTES | BIT MESRA | NEP SCHEME

Module 1: Lexical Analysis

Language Processors, Buffering, Thompson NFA & Direct DFA

Module I: Lexical Analysis & Automata Theory — Complete Syllabus Topics

1. Language Processors (Compiler, Interpreter, Assembler, Preprocessor, Linker, Loader)
2. 6 Phases of Compiler Architecture with Full Running Translation Traces
3. Compiler Front-End vs. Back-End & Pass Structures (1-Pass vs. Multi-Pass)
4. Lexical Analysis Role, Tokens, Patterns, Lexemes & Token Attributes
5. Input Buffering Mechanics (Two-Buffer Scheme & Sentinel Verification)
6. Regular Expressions & Regular Definitions for Programming Language Tokens
7. Thompson's Construction Algorithm (RE to ϵ-NFA with Structural Rules)
8. Subset Construction Algorithm (NFA to DFA Transition Table Derivations)
9. Direct DFA Construction from RE (Nullable, Firstpos, Lastpos, Followpos)
10. Hopcroft's DFA State Minimization Algorithm (Equivalence Partitioning Trace)
11. Lex & Flex Lexical Analyzer Generator Architecture & Ambiguity Rules
12. Comprehensive Solved BIT Mesra & GATE Question Bank (8 Solved Problems)

Topic 1: Language Processing Systems & Execution Pipeline

A **Compiler** is a sophisticated system software program that accepts a source program written in a high-level, human-readable programming language (such as C, C++, Rust, or Java) and translates it into an equivalent semantic representation in a low-level target language (machine code or assembly), while detecting and reporting all syntax and static semantic violations.

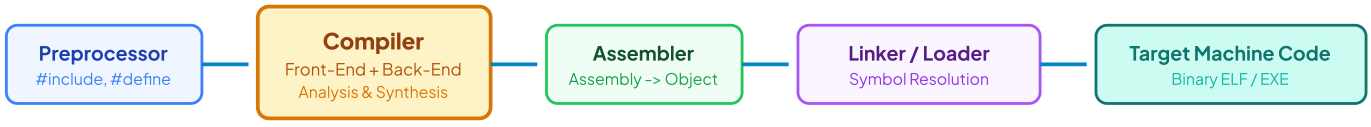

Figure 1.1: Complete End-to-End Language Processing Pipeline & System Tools

Detailed Analysis of System Components (The Cousins of the Compiler):

Tool	Input → Output	Primary Technical Responsibilities & Runtime Execution
1. Preprocessor	Raw High-Level Code → Pure High-Level Code	• **Macro Expansion:** Replaces macro identifiers with their replacement text (e.g., `#define PI 3.14159`). • **File Inclusion:** Injects verbatim contents of standard or user header files (e.g., `#include `). • **Conditional Compilation:** Evaluates compile-time conditionals (e.g., `#ifdef DEBUG ... #endif`) to strip unwanted source lines. • **Comment Stripping:** Removes all single-line `//` and block `/* ... */` comments from the source stream.
2. Compiler	Pure Source Code → Target Assembly Code	Performs full syntactic, semantic, and intermediate translation passes. Emits symbolic target assembly mnemonics (e.g., x86-64, ARM, RISC-V) rather than raw machine opcodes, allowing the assembler to handle platform-specific bit encodings.
3. Assembler	Assembly Mnemonics → Relocatable Object Code	Translates assembly instructions into raw binary machine opcodes. Constructs the initial **Relocatable Object File** (`.o` or `.obj`) containing machine instructions, initialized data tables, and an internal **Relocation Table** for external labels.
4. Linker	Relocatable Object Files + Static Libraries → Absolute Executable	• **Symbol Resolution:** Matches external function and variable references across separately compiled compilation units (e.g., linking `printf` calls to `libc.a`). • **Relocation:** Merges separate data and text sections into unified virtual address spaces, recalculating all relative memory offsets.
5. Loader	Absolute Executable on Disk → Running Process in RAM	Allocates primary memory space (Stack, Heap, Data, Text), loads machine instructions into RAM, sets up dynamic library linkages (`.so` / `.dll`), initializes hardware CPU registers (Instruction Pointer `RIP`, Stack Pointer `RSP`), and triggers execution.

Topic 2: The 6 Phases of Compiler Architecture

A modern compiler operates as a strict sequence of analytical (Front-End) and synthetic (Back-End) phases. Every phase transforms the program representation while maintaining bidirectional communication with the central **Symbol Table** and **Error Handler**.

🏛️ Complete End-to-End Tracing: Statement `position = initial + rate * 60`

To understand the continuous transformations, follow how the statement `position = initial + rate * 60` is processed across all 6 phases:

1. **Phase 1: Lexical Analysis (Scanning):**
   Reads raw character stream from left to right, partitions characters into valid lexemes, and emits a linear token sequence:

$$\langle \mathbf{id}, 1 \rangle \ \langle = \rangle \ \langle \mathbf{id}, 2 \rangle \ \langle + \rangle \ \langle \mathbf{id}, 3 \rangle \ \langle * \rangle \ \langle \mathbf{num}, 60 \rangle$$

   Symbol Table Entries: `1: position (float)`, `2: initial (float)`, `3: rate (float)`.

2. **Phase 2: Syntax Analysis (Parsing):**
   Verifies grammatical structure using Context-Free Grammars (CFG) and constructs a hierarchical **Abstract Syntax Tree (AST)** that inherently enforces operator precedence ($*$ binds tighter than $+$):

```
      =
     / \
id(1)   +
       / \
   id(2)   *
          / \
      id(3)  60
```

3. **Phase 3: Semantic Analysis (Type Checking & Coercion):**
   Checks language semantic constraints (variable declarations, scope, operand type compatibility). Since `rate` is a `float` and `60` is an `integer`, it inserts an implicit type conversion operator node $\mathrm{inttofloat}$:

```
      =
     / \
id(1)   +
       / \
   id(2)   *
          / \
      id(3)  inttofloat
                 |
                60
```

4. **Phase 4: Intermediate Code Generation (ICG):**
   Translates the decorated AST into machine-independent, linear **Three-Address Code (TAC)** where each instruction contains at most one operator:

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

5. **Phase 5: Machine-Independent Code Optimization:**
   Analyzes the TAC to eliminate redundant computations, evaluate constants at compile-time (Constant Folding), and reduce temporary register pressure:

```
t1 = id3 * 60.0        ; Constant folded inttofloat(60) → 60.0 at compile-time
id1 = id2 + t1         ; Eliminated unnecessary temporary t3
```

6. **Phase 6: Code Generation (Target Machine Assembly):**
   Maps intermediate variables to hardware CPU registers (`R1`, `R2`) and emits optimized target assembly instructions:

```
LDF    R2, id3          ; Load floating-point rate into register R2
MULF   R2, #60.0        ; Multiply R2 by constant literal 60.0
LDF    R1, id2          ; Load floating-point initial into register R1
ADDF   R1, R2           ; Add R2 into R1 (R1 = initial + rate * 60.0)
STF    id1, R1          ; Store result from R1 into memory location position
```

Topic 3: Compiler Passes & Architecture Organization

Front-End vs. Back-End Separation ($N \times M$ Retargetability Principle)

- **Front-End (Analysis):** Depends strictly on the source programming language (Lexical, Syntax, Semantic, and ICG). It produces a clean, machine-independent Intermediate Representation (IR).
- **Back-End (Synthesis):** Depends strictly on the target hardware architecture (Optimization, Register Allocation, Instruction Scheduling, Machine Code Generation).
- **Engineering Significance:** To build compilers for N high-level languages targeting M distinct CPU architectures, modular separation requires only N front-ends and M back-ends ($N + M$ modules), rather than $N \times M$ separate monolithic compilers!

Single-Pass vs. Multi-Pass Compilers:

Single-Pass Compiler: Reads source code once and generates target code directly without creating intermediate files. Highly memory efficient (used in early Pascal compilers), but cannot perform global optimizations or forward-referencing declarations without backpatching.
Multi-Pass Compiler: Traverses intermediate representations multiple times. Enables deep interprocedural optimizations, global register coloring, and dead code elimination at the cost of higher compilation latency and memory consumption.

Topic 4 & 5: Lexical Analysis Role, Token Definitions & Input Buffering

1. Essential Lexical Terminology:

Token: An abstract terminal category recognized by the compiler front-end (e.g., `KEYWORD`, `IDENTIFIER`, `NUMBER`, `RELOP`, `ASSIGN`).
Pattern: The formal grammatical specification (usually a Regular Expression) that character sequences must satisfy to match a token (e.g., `[a-zA-Z_][a-zA-Z0-9_]*`).
Lexeme: The concrete sequence of source characters that matches a token's pattern (e.g., `total_sum`, `3.14159`, `==`, `while`).
Token Attribute: Additional metadata stored alongside the token in the Symbol Table (e.g., type, memory offset, line number, scope depth).

2. Input Buffering Architecture (Two-Buffer Scheme with Sentinels):

Reading source files character-by-character via direct OS system calls incurs massive I/O overhead. A **Two-Buffer Scheme** allocates two contiguous N-byte blocks (typically $N = 4096$ bytes, matching the physical disk block size):

Figure 1.2: Two-Buffer Input Structure with Boundary Sentinel EOFs

Input Scanning Algorithm with Sentinel Checking:

```
switch (*forward++) {
    case EOF:
        if (forward is at end of first buffer half) {
            reload second buffer half;
            forward = beginning of second half;
        } else if (forward is at end of second buffer half) {
            reload first buffer half;
            forward = beginning of first half;
        } else {
            /* EOF is actual end of source file */
            terminate scanning;
        }
        break;
    /* Process regular characters without secondary boundary checks */
}
```

Topic 7: Thompson's Construction Algorithm (RE $\longrightarrow$ ϵ-NFA)

Thompson's Construction converts any Regular Expression r over alphabet Σ into an equivalent Non-Deterministic Finite Automaton with ϵ-transitions ($N(r)$) in linear time $O(|r|)$.

RE Expression	Automaton Topology & Transitions	Structural Invariant Properties
1. Basic Symbol ($a \in \Sigma$)	State $i \xrightarrow{a}$ State f	Exactly 1 start state, 1 accept state, no outgoing transitions from accept state.
2. Concatenation ($r_1 r_2$)	Accept state of $N(r_1)$ is merged with the start state of $N(r_2)$.	Initial start of $N(r_1)$ becomes overall start; accept of $N(r_2)$ becomes overall accept.
3. Alternation ($r_1 \mid r_2$)	New start state s_0 branches via ϵ to start of $N(r_1)$ and start of $N(r_2)$. Both accept states transition via ϵ to a new single accept state f_0.	Introduces 2 new states and 4 new ϵ-transitions.
4. Kleene Closure (r^*)	New start state s_0 branches via ϵ to start of $N(r)$ and directly to new accept state f_0 (for zero repetitions). Accept of $N(r)$ transitions via ϵ back to start of $N(r)$ and to f_0.	Introduces 2 new states and 4 new ϵ-transitions.

Topic 8: Subset Construction Algorithm (NFA $\longrightarrow$ DFA)

A Deterministic Finite Automaton (DFA) has no ϵ-transitions and has at most one outgoing edge per symbol from any state.

Formal Mathematical Operations in Subset Construction

- $\epsilon\text{-closure}(s)$: The set of all NFA states reachable from state s taking only ϵ-transitions:

$$\epsilon\text{-closure}(s) = \{s\} \cup \{t \mid \exists u \in \epsilon\text{-closure}(s) \text{ such that } u \xrightarrow{\epsilon} t\}$$

- $\epsilon\text{-closure}(T)$: $\bigcup_{s \in T} \epsilon\text{-closure}(s)$ for a set of states T.
- $\text{move}(T, a)$: Set of all NFA states reachable from any state in T on input symbol a:

$$\text{move}(T, a) = \{t \mid \exists s \in T \text{ such that } s \xrightarrow{a} t\}$$

🏛️ Step-by-Step Solved Problem: Convert $(a \mid b)^*abb$ to DFA via Subset Construction

Given NFA State Graph: Start state 0, accept state 10.

Step 1: Compute Initial DFA State $A = \epsilon\text{-closure}(0) = \{0, 1, 2, 4, 7\}$.

Step 2: Iteratively Compute Transitions for All Unmarked DFA States:

DFA State	NFA State Subset	Input 'a'	Input 'b'
A (Start)	$\{0, 1, 2, 4, 7\}$	$\mathbf{B} = \{1, 2, 3, 4, 6, 7, 8\}$	$\mathbf{C} = \{1, 2, 4, 5, 6, 7\}$
B	$\{1, 2, 3, 4, 6, 7, 8\}$	$\mathbf{B} = \{1, 2, 3, 4, 6, 7, 8\}$	$\mathbf{D} = \{1, 2, 4, 5, 6, 7, 9\}$
C	$\{1, 2, 4, 5, 6, 7\}$	$\mathbf{B} = \{1, 2, 3, 4, 6, 7, 8\}$	$\mathbf{C} = \{1, 2, 4, 5, 6, 7\}$
D	$\{1, 2, 4, 5, 6, 7, 9\}$	$\mathbf{B} = \{1, 2, 3, 4, 6, 7, 8\}$	$\mathbf{E}^* = \{1, 2, 4, 5, 6, 7, 10\}$
E* (Accept)	$\{1, 2, 4, 5, 6, 7, 10\}$	$\mathbf{B} = \{1, 2, 3, 4, 6, 7, 8\}$	$\mathbf{C} = \{1, 2, 4, 5, 6, 7\}$

Conclusion: The resulting DFA has 5 states $\{A, B, C, D, E\}$ with E as the unique accepting state.

Topic 9: Direct DFA Construction from Regular Expressions

Direct DFA construction bypasses NFA intermediate generation by decorating the **Augmented Syntax Tree** $(r)\#$ with structural position functions:

Node Type n	$\text{nullable}(n)$	$\text{firstpos}(n)$	$\text{lastpos}(n)$
ϵ-leaf	true	$\emptyset$	$\emptyset$
Leaf position i	false	$\{i\}$	$\{i\}$
$c_1 \mid c_2$ (Or)	$\text{nullable}(c_1) \vee \text{nullable}(c_2)$	$\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$	$\text{lastpos}(c_1) \cup \text{lastpos}(c_2)$
$c_1 \cdot c_2$ (Cat)	$\text{nullable}(c_1) \wedge \text{nullable}(c_2)$	if $\text{nullable}(c_1)$ then $\text{firstpos}(c_1) \cup \text{firstpos}(c_2)$ else $\text{firstpos}(c_1)$	if $\text{nullable}(c_2)$ then $\text{lastpos}(c_1) \cup \text{lastpos}(c_2)$ else $\text{lastpos}(c_2)$
c_1^* (Star)	true	$\text{firstpos}(c_1)$	$\text{lastpos}(c_1)$

Rules for Computing $\text{followpos}(i)$:

1. If n is a Cat-node with left child c_1 and right child c_2, for every position $i \in \text{lastpos}(c_1)$, all positions in $\text{firstpos}(c_2)$ are in $\text{followpos}(i)$.
2. If n is a Star-node with child c_1, for every position $i \in \text{lastpos}(c_1)$, all positions in $\text{firstpos}(c_1)$ are in $\text{followpos}(i)$.

Topic 10: Hopcroft's DFA Minimization Algorithm

Hopcroft's Algorithm computes the unique minimal-state DFA in $O(k \cdot n \log n)$ time by iteratively partitioning states into behavioral equivalence classes:

Initial Partition (P_0): Split all states into Accept (F) and Non-Accept ($S \setminus F$) states:

$$P_0 = \{F,\ S \setminus F\}$$

Refinement: For each group $G \in P$ and symbol $a \in \Sigma$, if $\text{move}(s, a)$ leads states of G into different partition groups, split G into separate sub-groups.
Fixed-Point Convergence: Terminate when $P_{k+1} = P_k$. Collapse each partition group into a single state in the minimal DFA.

Topic 11: Lex & Flex Lexical Analyzer Generator Specifications

Lex Section	Syntax & Contents	Key Lex Variables & Routines
1. Definitions	C headers, `#include`, `%option noyywrap`, regular definitions (`digit [0-9]`).	`yyin` (input file stream pointer), `yyout` (output stream pointer).
2. Translation Rules	Pairs of Regular Expressions and C action blocks: `{digit}+ { yylval = atoi(yytext); return NUM; }`	`yytext` (pointer to matched lexeme string), `yyleng` (length of matched lexeme), `yylval` (semantic attribute value).
3. User Subroutines	Auxiliary C functions, `main()` driver, `yywrap()` handler.	`yylex()` (main DFA scanning function returning integer token codes).

Lex Ambiguity Resolution Rules

1. **Longest Match (Maximal Munch):** The lexer always picks the rule that matches the longest possible sequence of characters (e.g., `<=` is matched as `LE` rather than `<` followed by `=`).
2. **First Match Rule:** If multiple regular expressions match the exact same longest prefix, Lex selects the rule that appears earliest in the `.l` specification file (e.g., `if` keyword listed before generic identifier `[a-z]+`).

🏛️ Top BIT Mesra Exam Questions & Answers (Module I)

Q1. Explain the role of Lexical Analyzer and state 4 reasons why it is separated from the Parser. (8 Marks)

1. **Simplicity of Compiler Architecture:** Separating character-level lexical processing from grammatical phrase parsing simplifies both stages. The parser processes abstract token streams without handling whitespace, tabs, and comments.
2. **Compiler Efficiency:** Specialized high-performance input buffering techniques can be applied to the lexical scanner. Over 70% of compilation CPU cycles are spent scanning characters.
3. **Compiler Portability:** Character-set dependencies (ASCII, UTF-8, Unicode) are isolated strictly inside the lexical phase, making the parser independent of source encoding.
4. **Modular Grammar Design:** Language syntax rules can focus purely on grammar hierarchies rather than token spelling.

Q2. Differentiate between Compiler and Interpreter across 5 core engineering parameters. (6 Marks)

1. **Translation Mechanism:** Compiler translates entire source code into native machine code before execution; Interpreter translates and executes line-by-line at runtime.
2. **Execution Speed:** Compiled machine code executes 10–50× faster than interpreted bytecode.
3. **Memory Usage:** Compiler requires memory for object code storage; Interpreter requires memory for runtime engine.
4. **Error Reporting:** Compiler reports all syntax/semantic errors collectively during compilation; Interpreter stops at the first runtime error encountered.
5. **Examples:** C, C++, Rust (Compiled); Python, Ruby, PHP (Interpreted).

Q3. Trace the Direct DFA construction for regular expression $(a \mid b)^* abb\#$. (10 Marks)

1. **Augmented Syntax Tree:** Leaf positions: $1 : a, 2 : b, 3 : a, 4 : b, 5 : b, 6 : \#$.
2. **Firstpos / Lastpos:** $\text{firstpos}(\text{root}) = \{1, 2, 3\}$.
3. **Followpos Table:** - $\text{followpos}(1) = \{1, 2, 3\}$ - $\text{followpos}(2) = \{1, 2, 3\}$ - $\text{followpos}(3) = \{4\}$ - $\text{followpos}(4) = \{5\}$ - $\text{followpos}(5) = \{6\}$ 4. **DFA States:** State $A = \{1, 2, 3\}$; State $B = \{1, 2, 3, 4\}$; State $C = \{1, 2, 3, 5\}$; State $D^* = \{1, 2, 3, 6\}$ (Accept).